



GUIA VISUAL EXPLICATIVO

Como Cada Projeto Funciona

Diagramas de arquitetura, fluxogramas e explicação
passo a passo dos 7 projetos de Apache Kafka

Partições • Consumer Groups • Chaves e Ordenação
Retry Manual e DLQ • Transações Exactly-Once
Do desenho da arquitetura ao passo a passo de execução

Sumário

- 0. Como ler os diagramas deste guia
- 1. Projeto 1 — Hello World
- 2. Projeto 2 — Partições e Consumer Group
- 3. Projeto 3 — Múltiplos Consumer Groups Independentes
- 4. Projeto 4 — Chaves e Ordenação
- 5. Projeto 5 — Falhas, Retry e Reprocessamento
- 6. Projeto 6 — Exactly-Once com Transações
- 7. Projeto 7 — Consumer Produtivo
- 8. Visão comparativa dos 7 projetos

0. Como ler os diagramas deste guia

Cada projeto tem três elementos visuais diferentes, que se complementam:

Elemento	O que mostra
Diagrama de arquitetura	As "peças" envolvidas (producer, tópico, partições, consumer groups) e como estão conectadas — a "planta baixa" do sistema.
Fluxograma numerado	A ordem exata dos acontecimentos, passo a passo, como uma receita.
Tabela de decisão (quando aplicável)	O que acontece em cada cenário possível — sucesso, falha, timeout etc.

Sugestão de uso

Leia primeiro o diagrama de arquitetura para entender "quem é quem". Depois siga o fluxograma numerado acompanhando o código do PDF de projetos práticos, linha por linha, vendo onde cada passo do diagrama aparece no código Java.

O que muda em relação ao guia visual de RabbitMQ

No Kafka, a peça central de quase todo diagrama é a **partição**, não a fila. Preste atenção especial em como cada projeto desenha as partições como "trilhas" paralelas — é a chave visual para entender paralelismo, ordenação e consumer groups ao mesmo tempo.

1. Projeto 1 — Hello World

PROJETO 1 · PRODUTOR/CONSUMIDOR BÁSICO · 1 PARTIÇÃO

pedido-hello-world

1.1. O que este projeto prova

A diferença mais fundamental em relação ao RabbitMQ: aqui não existe "fila que esvazia". O producer escreve em um **log append-only** (a partição), e o consumer avança um marcador (offset) por ele — sem remover nada. O objetivo deste primeiro projeto é só sentir esse modelo mental antes de qualquer complexidade extra.

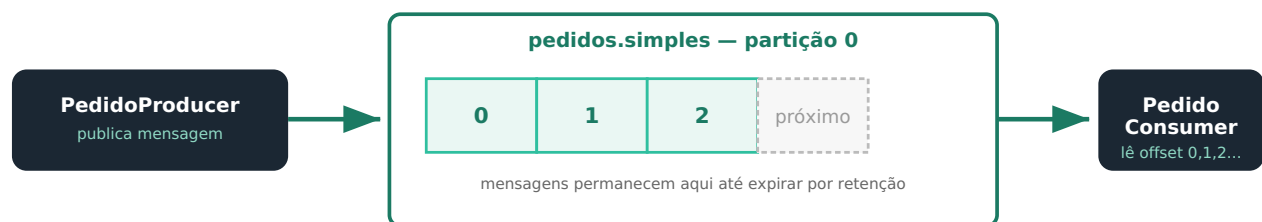


Fig. 1.1 — O producer escreve no final do log (append-only). O consumer lê avançando um marcador (offset), sem remover nada da partição.

1.2. Fluxograma passo a passo

- 1 PedidoProducer abre uma conexão com o broker (`bootstrap.servers=localhost:9092`).
- 2 Publica um `ProducerRecord` no tópico `pedidos.simples` . Sem chave informada, cai na única partição existente (partição 0).
- 3 O Kafka anexa a mensagem no final do log da partição, atribuindo o próximo offset disponível (0, depois 1, depois 2...).
- 4 PedidoConsumer se conecta usando um `group.id` e faz `subscribe` no mesmo tópico.
- 5 Em loop, chama `poll()` — o Kafka entrega os registros a partir do offset onde o consumer parou da última vez (ou do início, se nunca leu nada e `auto.offset.reset=earliest`).
- 6 O consumer imprime a mensagem, incluindo partição e offset recebidos.
- 7 A mensagem **continua existindo** na partição — nada foi removido. Se você rodar outro consumer com um `group.id` diferente, ele pode ler a mesma mensagem do zero.

O que observar na prática

Publique 3 mensagens seguidas e note os offsets 0, 1, 2 no console do consumer. Depois, mate o consumer e publique mais 2 mensagens (offsets 3 e 4). Ao religar o consumer com o **mesmo** `group.id` , ele retoma exatamente do offset 3 — não do início, e não perde nada. Esse comportamento (retomar de onde parou, por grupo) é a base de tudo que vem nos próximos projetos.

2. Projeto 2 — Partições e Consumer Group

PROJETO 2 · MÚLTIPLAS PARTIÇÕES · PARALELISMO REAL

gerador-relatorios

2.1. O que este projeto prova

Aqui o tópico tem 4 partições, e 3 instâncias do mesmo consumer (mesmo `group.id`) competem pela leitura. Diferente do Work Queue do RabbitMQ — onde qualquer worker pode pegar a próxima mensagem disponível — no Kafka o Kafka **atribui partições inteiras** a cada consumer. O paralelismo máximo é sempre limitado ao número de partições, não ao número de workers.

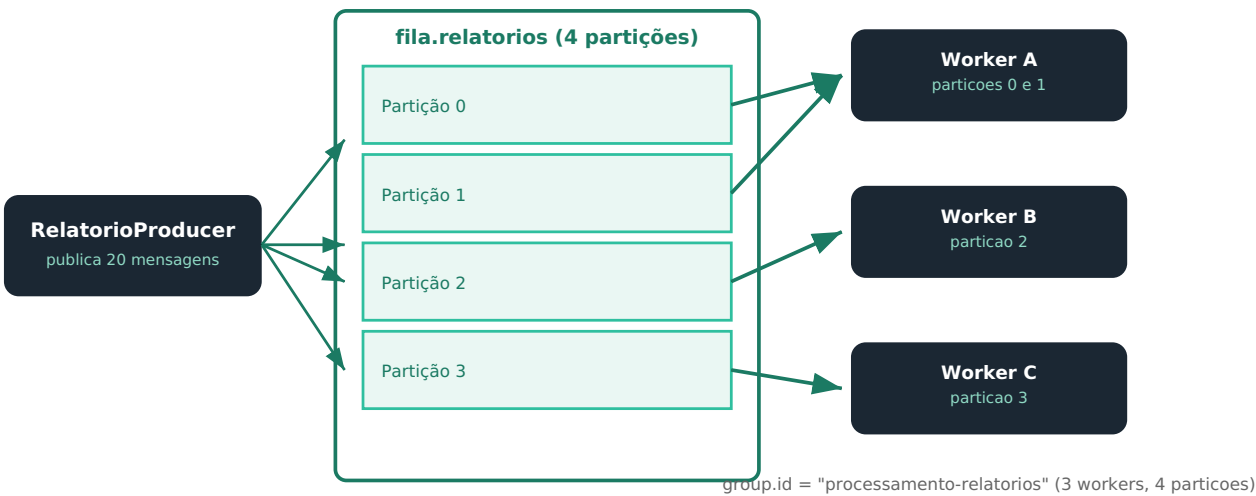


Fig. 2.1 — O Kafka atribui partições inteiras a cada worker. Com 4 partições e 3 workers, um deles fica com 2 partições — nunca existe divisão "parcial" de uma partição.

2.2. Fluxograma passo a passo

- Três instâncias de `RelatorioWorker` sobem, todas com o mesmo `group.id="processamento-relatorios"`, e fazem `subscribe` no tópico.
- O Kafka executa um **rebalance**: distribui as 4 partições entre os 3 workers ativos (ex.: Worker A fica com as partições 0 e 1; B com a 2; C com a 3).
- `RelatorioProducer` publica 20 mensagens sem chave — o Kafka as distribui entre as 4 partições via sticky partitioning.
- Cada worker só recebe, via `poll()`, registros das partições que lhe foram atribuídas — nunca das partições de outro worker do mesmo grupo.
- Cada worker processa e confirma (`commitSync()`) seu próprio progresso, de forma independente por partição.
- Se um worker cair, o Kafka detecta (via heartbeat) e dispara um novo rebalance, redistribuindo as partições órfãs entre os workers restantes.

Cenário	O que acontece
3 workers, 4 partições	Distribuição desigual: um worker fica com 2 partições, os outros com 1 cada
4 workers, 4 partições	Distribuição perfeita: 1 partição por worker
5 workers, 4 partições	1 worker fica ocioso — não existe partição sobrando para ele
Um worker cai	Rebalance automático — suas partições são redistribuídas entre os workers restantes

Diferente do RabbitMQ

No Work Queue de RabbitMQ, adicionar mais um worker sempre ajuda (ele passa a competir pela próxima mensagem disponível). Aqui, adicionar mais workers do que partições não aumenta o paralelismo — os excedentes simplesmente ficam sem nada para fazer até que uma partição nova seja criada ou outro worker caia.

3. Projeto 3 — Múltiplos Consumer Groups Independentes

PROJETO 3 · FANOUT NATIVO · OFFSETS INDEPENDENTES

pedido-criado-eventos

3.1. O que este projeto prova

Três serviços diferentes (Email, Estoque, Analytics) precisam receber **o mesmo evento completo**, cada um no seu próprio ritmo. No RabbitMQ isso exigia uma exchange fanout com 3 filas fisicamente separadas. No Kafka, basta que cada serviço use um `group.id` diferente — o mesmo tópico, a mesma partição física, é lida três vezes de forma totalmente independente.

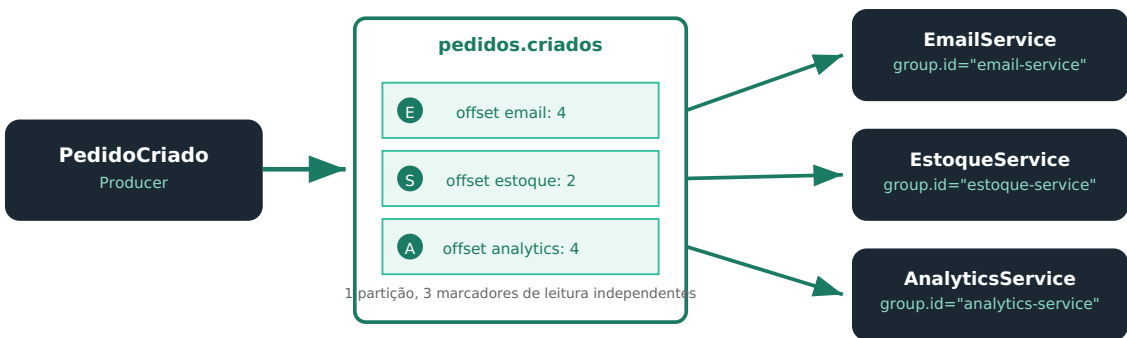


Fig. 3.1 — A mesma partição física é lida por três consumer groups diferentes, cada um com seu próprio offset — sem duplicar dados fisicamente, ao contrário de 3 filas separadas no RabbitMQ.

3.2. Fluxograma passo a passo

- 1 Os três serviços sobem, cada um com um `group.id` diferente, e fazem `subscribe` no mesmo tópico `pedidos.criados`.
- 2 O Kafka cria (ou reaproveita) três marcadores de offset independentes — um por grupo — no tópico interno `__consumer_offsets`.
- 3 `PedidoCriadoProducer` publica um evento uma única vez.
- 4 O Kafka **não duplica** o dado fisicamente — a mensagem é escrita uma vez na partição.
- 5 Cada um dos três consumer groups, de forma independente e no seu próprio ritmo, lê essa mesma mensagem e avança seu próprio offset.
- 6 Se o `AnalyticsService` estiver mais lento (ou desligado por um tempo), isso não afeta em nada a velocidade de leitura do `EmailService` ou do `EstoqueService`.

O replay como consequência natural

Como cada grupo mantém seu próprio offset, um **novo** consumer group (nunca visto antes pelo Kafka) sempre pode nascer lendo o tópico inteiro desde o começo (`auto.offset.reset=earliest`) — mesmo que os eventos tenham sido publicados há semanas. É assim que um serviço novo pode ser populado com o histórico completo de eventos, algo que não existe no RabbitMQ.

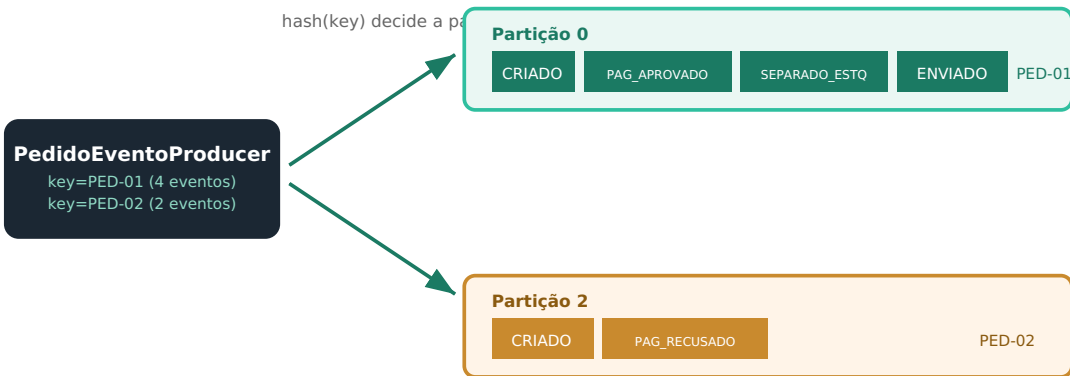
4. Projeto 4 — Chaves e Ordenação

PROJETO 4 · PARTICIONAMENTO POR CHAVE · GARANTIA DE ORDEM

pedido-eventos-por-cliente

4.1. O que este projeto prova

A ordem só é garantida **dentro da mesma partição**. E qual partição uma mensagem recebe é decidido, por padrão, pelo hash da **chave**. Esse projeto mostra visualmente por que eventos do mesmo pedido (mesma chave) sempre ficam em ordem, enquanto eventos sem chave podem "se espalhar" e chegar fora de ordem entre si.



Mesma chave = mesma partição = ordem garantida entre si. Chaves diferentes: sem relação de ordem.

Fig. 4.1 — Todos os 4 eventos de PED-01 caem na Partição 0, na ordem exata em que foram publicados. Os eventos de PED-02, com chave diferente, caem em outra partição — sem garantia de ordem relativa entre os dois pedidos.

4.2. Fluxograma passo a passo

- 1 Producer publica 4 eventos com `key="PED-01"`, em sequência: CRIADO, PAGAMENTO_APROVADO, SEPARADO_ESTOQUE, ENVIADO.
- 2 Para cada publicação, o Kafka calcula `hash("PED-01") % número_de_partições` — o resultado é sempre o mesmo número, então todos os 4 eventos vão para a mesma partição.
- 3 Dentro dessa partição, os eventos ficam gravados na ordem exata em que chegaram — offset 0, 1, 2, 3.
- 4 Producer publica 2 eventos com `key="PED-02"` — o hash é diferente, então (na maioria dos casos) caem em uma partição diferente.
- 5 Um consumer lendo essa partição recebe os eventos de PED-01 sempre na ordem CRIADO → APROVADO → SEPARADO → ENVIADO — nunca fora dessa ordem.
- 6 Não existe, porém, nenhuma garantia sobre a ordem relativa entre eventos de PED-01 e PED-02 — eles vivem em "trilhas" (partições) diferentes e independentes.

Cenário	Garantia de ordem
Mesma chave, mesmo tópico	✓ Garantida — sempre mesma partição, ordem de publicação preservada
Chaves diferentes, mesmo tópico	✗ Nenhuma garantia entre si — podem cair em partições diferentes
Sem chave (key=null)	✗ Nenhuma garantia — distribuição por sticky partitioning, sem relação com conteúdo

Erro comum

Achar que "publiquei A antes de B, então A sempre chega antes" vale sempre no Kafka. Isso só é verdade se A e B tiverem a **mesma chave**. É o erro mais comum de quem vem de sistemas com uma única fila (como o RabbitMQ), onde a ordem de uma fila é naturalmente total.

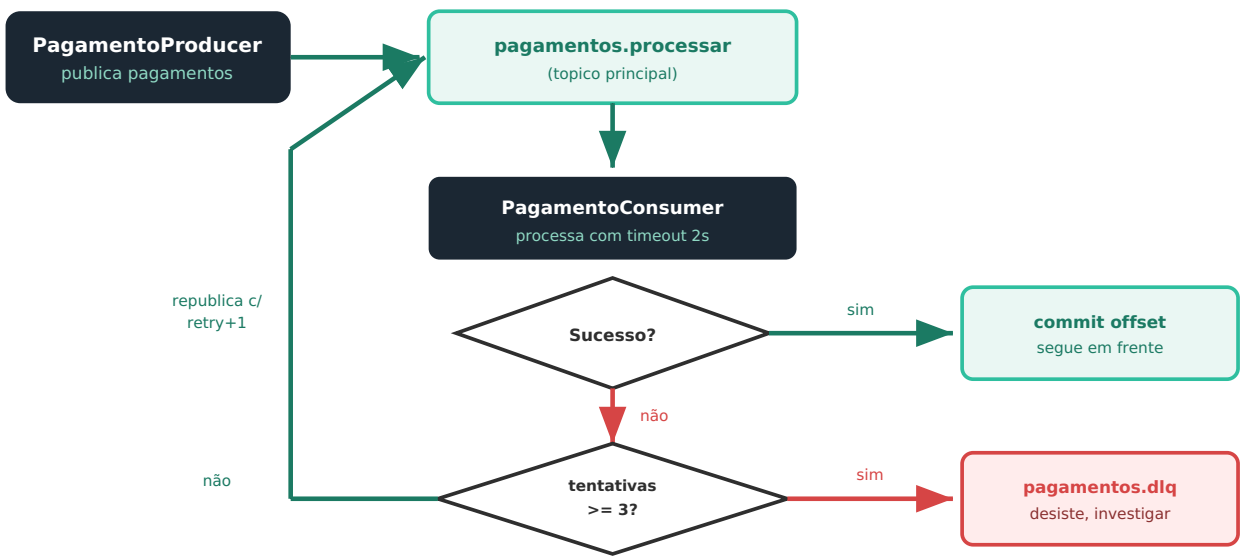
5. Projeto 5 — Falhas, Retry e Reprocessamento

PROJETO 5 · DLQ MANUAL · BACKOFF CONTROLADO PELO CÓDIGO

pagamento-com-retry

5.1. O que este projeto prova

Diferente do RabbitMQ (onde uma fila de espera com TTL cronometra o atraso automaticamente), o Kafka não tem nenhum mecanismo nativo de dead-lettering. Este projeto mostra como implementar retry com limite de tentativas e uma "DLQ manual" — dois tópicos comuns e um pouco de lógica no consumer, sem nenhum recurso especial do broker.



O backoff (espera entre tentativas) é feito com `Thread.sleep` no código — não existe TTL automático como no RabbitMQ.

Fig. 5.1 — O ciclo de retry inteiro é orquestrado pelo próprio consumer: republicar no mesmo tópico com contador incrementado, ou desviar para a DLQ manual após o limite.

5.2. Fluxograma de decisão por mensagem

- 1 PagamentoConsumer recebe um registro de `pagamentos.processar` e lê o header `x-retry-count` (0 se ausente).
- 2 Processa com timeout de 2s, usando structured concurrency (Java 25) — se demorar mais que isso, é tratado como falha.
- 3 Sucesso? → `commitSync()` — offset confirmado, mensagem nunca mais revisitada.
- 4 Falha e tentativas ainda não atingiram 3? → aguarda um tempo fixo (`Thread.sleep`) e republica no **mesmo tópico** `pagamentos.processar`, com o header `x-retry-count` incrementado.
- 5 Falha e já é a 3ª tentativa? → publica o registro original em `pagamentos.dlq`, um tópico separado, só para investigação manual.
- 6 Em qualquer um dos dois casos de falha, o offset da mensagem original também é confirmado — ela não fica "presa" retornando ao mesmo offset repetidamente; uma **cópia nova** dela é quem carrega a tentativa seguinte.

Diferença crucial em relação ao RabbitMQ

No RabbitMQ, a mensagem "original" circula fisicamente entre filas até ser confirmada ou descartada. No Kafka, cada tentativa de retry é uma **mensagem nova**, publicada pelo próprio consumer, carregando o mesmo payload e um contador atualizado — o registro original no tópico principal já foi confirmado (commitado) e nunca mais será lido

novamente por este consumer group.

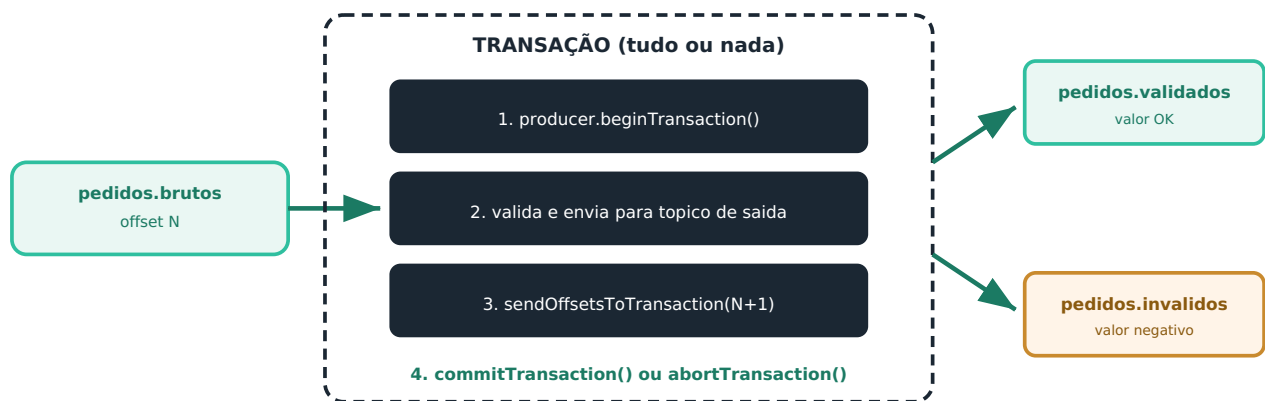
6. Projeto 6 — Exactly-Once com Transações

PROJETO 6 · READ-PROCESS-WRITE ATÔMICO

pedido-validador-transacional

6.1. O que este projeto prova

Este é o projeto sem equivalente direto no guia de RabbitMQ. Ele mostra como o Kafka garante que **ler de um tópico, escrever em outro, e confirmar o offset de leitura** aconteçam como uma única operação atômica — tudo ou nada — mesmo que o processo caia no meio do caminho.



Se o processo cair antes do commit, a transação inteira é abortada — nada é escrito, o offset não avança.

Fig. 6.1 — A escrita no tópico de destino e o avanço do offset de leitura acontecem dentro da mesma transação: ou os dois acontecem, ou nenhum dos dois acontece.

6.2. Fluxograma passo a passo

- 1 Validador chama `producer.initTransactions()` uma única vez, no início, usando um `transactional.id` fixo e estável para esta instância.
- 2 A cada lote de registros lidos (`poll`), abre uma transação com `beginTransaction()` .
- 3 Para cada registro do lote, valida a regra de negócio e publica no tópico de destino apropriado (`pedidos.validados` ou `pedidos.invalidos`) — ainda dentro da transação, nada é visível para fora ainda.
- 4 Registra, também dentro da mesma transação, o novo offset de leitura a ser confirmado, via `sendOffsetsToTransaction()` .
- 5 Se tudo correu bem, `commitTransaction()` torna as escritas e o commit de offset visíveis atomicamente — de uma vez só.
- 6 Se qualquer exceção acontecer no meio do processo, `abortTransaction()` descarta tudo — nenhuma escrita parcial fica visível, e o offset de leitura não avança (a próxima chamada de `poll` vai reler o mesmo lote).

O papel do `isolation.level=read_committed`

Consumers que leem os tópicos de destino (`pedidos.validados`) precisam configurar `isolation.level=read_committed` para nunca enxergar dados de uma transação que foi abortada. Sem essa configuração (o padrão é `read_uncommitted`), um consumer poderia ler um registro "fantasma" que tecnicamente nunca foi confirmado.

7. Projeto 7 — Consumer Produtivo

PROJETO 7 · IDEMPOTÊNCIA POR (PARTIÇÃO, OFFSET) · WAKEUP

estoque-consumer-robusto

7.1. O que este projeto prova

Fecha o ciclo com as mesmas duas preocupações de produção do Projeto 7 de RabbitMQ, mas usando os mecanismos idiomáticos do Kafka: idempotência via o identificador natural `(partição, offset)`, e desligamento seguro via `consumer.wakeup()` — mais simples do que o enum de estados manual usado no RabbitMQ.

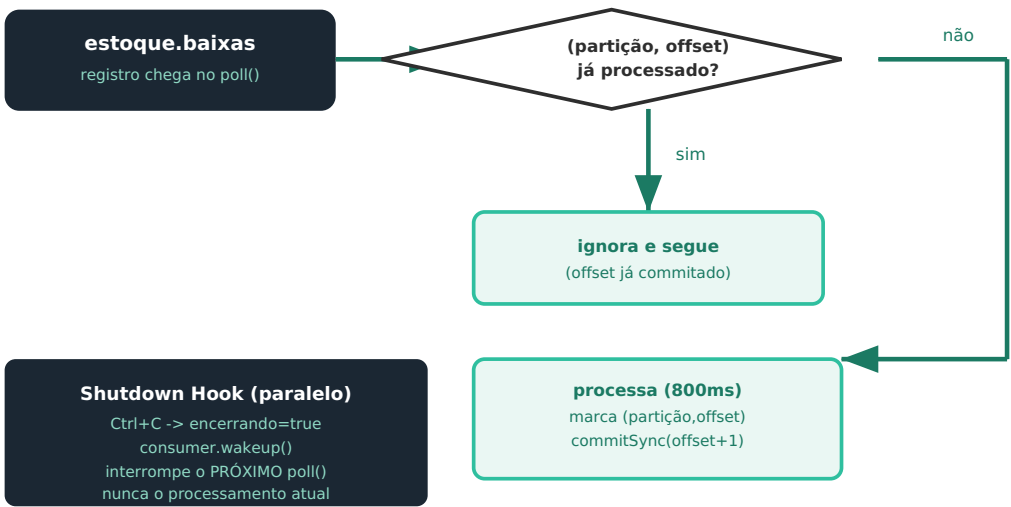


Fig. 7.1 — A checagem de idempotência usa o identificador natural do Kafka; o shutdown gracioso interrompe apenas a espera por novos registros, nunca um processamento já em andamento.

7.2. Fluxograma passo a passo

- 1 Um registro chega pelo `poll()` do loop principal.
- 2 O consumer verifica se o par `(partição, offset)` desse registro já foi processado antes (`RegistroIdempotencia`).
- 3 Se já foi, simplesmente ignora e segue para o próximo registro do lote — sem reprocessar, sem novo commit.
- 4 Se é novo, processa de fato (simulado com 800ms), marca o par como processado, e confirma `commitSync` imediatamente após — offset por offset, não em lote.
- 5 Em paralelo, a qualquer momento, um `Ctrl+C` dispara o shutdown hook, que muda a flag `encerrando` para `true` e chama `consumer.wakeup()`.
- 6 `wakeup()` faz a próxima chamada de `poll()` lançar `WakeupException` — mas nunca interrompe um processamento que já estava em andamento dentro do laço `for`.
- 7 A exceção é capturada, o loop termina, e `consumer.close()` libera a instância do grupo de forma limpa, permitindo que outra instância assuma suas partições rapidamente.

Mecanismo	RabbitMQ (Projeto 7)	Kafka (este projeto)
Identificador de	<code>message_id</code> definido manualmente pelo producer	<code>(partição, offset)</code> — natural, atribuído pelo broker

idempotência		
Sinalização de shutdown	Enum de estados (<code>RODANDO</code> / <code>DRENANDO</code> / <code>PARADO</code>) verificado a cada mensagem	<code>consumer.wakeup()</code> interrompe apenas a espera por novos dados
O que acontece com o "em andamento"	Shutdown hook espera a flag <code>processandoAgora</code> voltar a <code>false</code>	Processamento em curso nunca é interrompido — <code>wakeup()</code> só afeta o próximo <code>poll()</code>

Por que o Kafka é mais simples aqui

Como o `poll()` e o processamento do lote acontecem sequencialmente na mesma thread (diferente do callback assíncrono do RabbitMQ), não existe risco de uma nova mensagem chegar "no meio" de um processamento em curso. Isso elimina a necessidade do `AtomicBoolean processandoAgora` e do enum de estados — bastou verificar uma flag simples no início de cada iteração do laço principal.

8. Visão comparativa dos 7 projetos

Depois de estudar cada um isoladamente, vale enxergar a progressão como um todo — e, principalmente, comparar diretamente com a jornada equivalente que você já fez em RabbitMQ.

#	Projeto Kafka	Conceito central	Equivalente em RabbitMQ
1	Hello World	Producer/consumer básico, 1 partição	Projeto 1 — Hello World
2	Partições e Consumer Group	Paralelismo limitado ao nº de partições	Projeto 2 — Work Queue
3	Múltiplos Consumer Groups	Fanout nativo + replay de histórico	Projeto 3 — Fanout
4	Chaves e Ordenação	Ordem garantida só dentro da partição	Projeto 4 — Topic Exchange (mecanismo diferente)
5	Falhas, Retry e Reprocessamento	DLQ manual, backoff no código	Projeto 5 — DLX + Retry (nativo)
6	Exactly-Once com Transações	Read-process-write atômico	Sem equivalente direto
7	Consumer Produtivo	Idempotência + wakeup shutdown	Projeto 7 — Consumer Robusto

8.1. Mapa mental de decisão

? Preciso de paralelismo real dividindo o trabalho entre várias instâncias? → Múltiplas partições + Consumer Group (Projeto 2)

? Preciso que times/serviços diferentes leiam o mesmo fluxo de eventos, cada um no seu próprio ritmo? → Múltiplos Consumer Groups (Projeto 3)

? A ordem entre eventos relacionados importa? → Publique sempre com a mesma chave (Projeto 4)

? Meu processamento pode falhar e preciso de um caminho de erro? → DLQ manual + retry (Projeto 5) — não é automático como no RabbitMQ

? Preciso que ler-transformar-escrever aconteça como uma unidade atômica? → Transações (Projeto 6)

? Meu consumer vai rodar em produção de verdade? → Idempotência por (partição, offset) + wakeup (Projeto 7) — em qualquer um dos padrões anteriores

A maior lição de todo o guia

Se tem uma ideia para levar consigo depois destes 7 projetos, é esta: no RabbitMQ, o broker faz o trabalho pesado de roteamento, retry e dead-lettering — você configura, e ele executa. No Kafka, o broker é "burro" de propósito (só armazena e serve um log ordenado) — quase todo esse tipo de lógica de negócio (retry, DLQ, idempotência) mora no **seu código**, não no broker. Essa é a troca fundamental entre os dois modelos.

Próximo passo

Com os 7 projetos de Kafka rodando e este guia visual como referência, você completa o mesmo ciclo de aprendizado que já fez com RabbitMQ — teoria, prática, e entendimento visual — para as duas tecnologias de mensageria mais usadas em arquiteturas de microsserviços atualmente. Os próximos passos naturais, se quiser aprofundar ainda mais, seriam explorar o Kafka Streams para processamento contínuo, ou montar um projeto que combine as duas ferramentas — um cenário comum em arquiteturas reais.